

Biomedical Sensors and Signal Processing Laboratory 1

Data acquisition channel and post-processing

Outline of the document:

1. Previous work.
 2. Lab work.
 3. The report.
 4. Good practice code.
- References and helping

The objective of this lab is to understand the data acquisition channel operation and the subsequent post-processing that must be done by the processor.

In the data acquisition channel, a physical parameter is measured. The data is conditioned and sent to the processor. The processor's ADC digitizes the signal. The processor is responsible for calculating the value of the detected physical parameter. An LCD display shows the information.

The circuit design corresponds to the typical data acquisition channel explained in class. It works with the classical Arduino Uno development board. The board is based on an ATmega128 microcontroller (Fig. 1) which is responsible to perform all the processing.

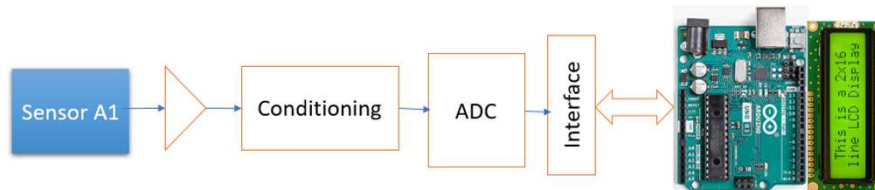


Fig. 1. Digital temperature thermometer.

Several software tools are available to design and simulate the data acquisition channel. The most important ones are paid and usually they need of a certain learning curve. However, to design the circuit that models the data acquisition channel, it is sufficient to work with *Autodesk Tinkercad*, a free and easy-to-use mixed-signal software tool.

1. Previous work

The previous work is about familiarizing yourself with the software. The first step is to create a personal account in *Autodesk Tinkercad*,

<https://www.tinkercad.com/>

Once created it is recommended to take a tour of its tabs:

- In the upper right you can see that there is a tab to sign up. It is recommended to do so, since this way you have access to the repository of designs that we can make.

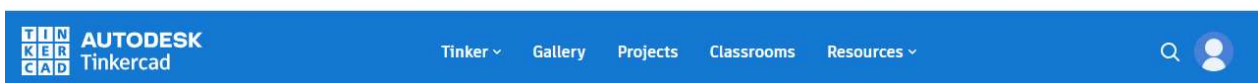


Fig. 2. Autodesk Tinkercad tab.

- Once done, you can take a tour though the central tabs: *Tinker*, *Gallery*, *Projects*, *Classrooms* and *Resources*. The most important ones for us are *Tinker* and *Resources*.

The *Tinker* tab shows the different types of applications that can be made. We will be interested in *Circuits*. This is where we can come in to realize our designs. For programming you could also use the *Codeblocks* tab. But for our more electronic designs, and working with Arduino, it will be better if we program directly with Arduino's C++.

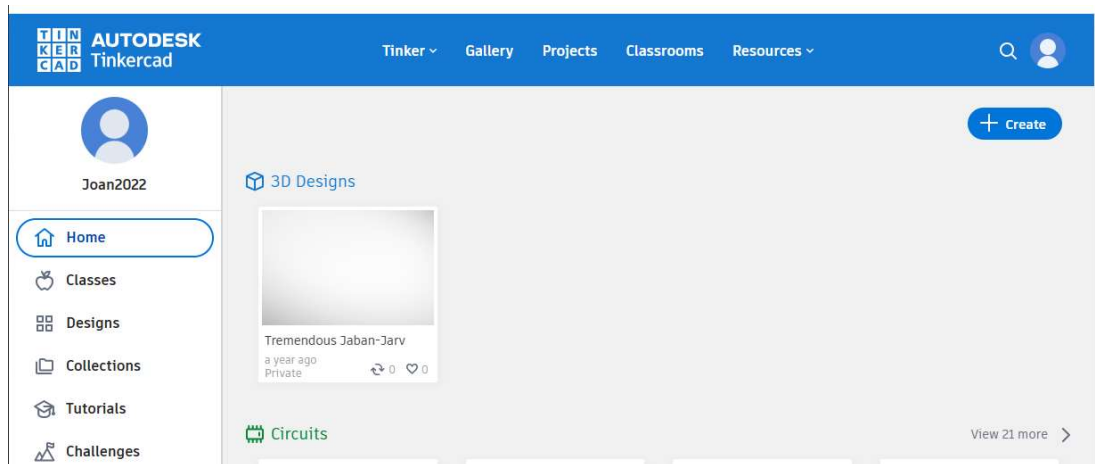


Fig. 3. The circuit is introduced using the *Circuits* design platform.

- In the *Resources* tab we will find application examples. In the *Learning centre* and under *Learn circuits* there are some guided examples that allow you to get started in electronics (Fig. 4).

Learn Circuits

These starter projects are the perfect launchpad to all things Tinker.

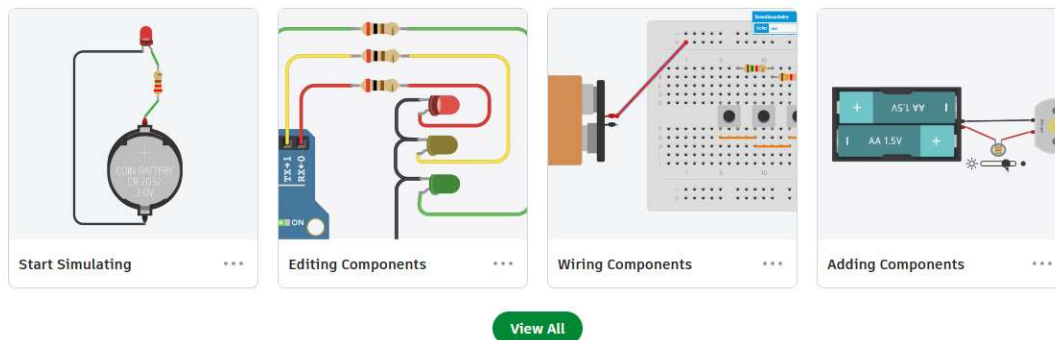


Fig. 4. *Learn circuits* page.

All the design we make are stored in *Your designs* and in the category *Circuits*.

The designs are also accessed clicking in the *Autodesk Tinkercad* icon (Fig. 5).



Fig. 5. Autodesk Tinkercad icon to access to designs.

To start a new design, simply go to *Design* and click *Create* for a new *Circuit* (Fig. 6).

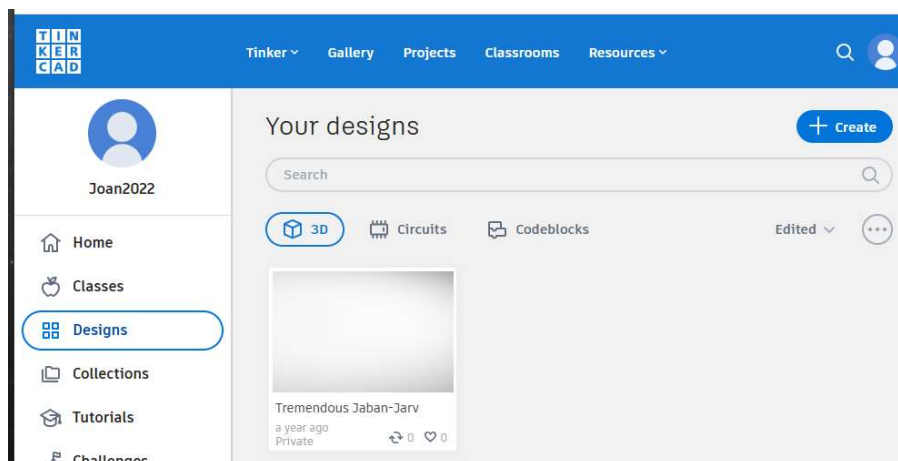


Fig. 6. Start a new design.

2. Lab work.

As introduced, the main objective of this lab is to understand the data acquisition channel design. Several sensors can be used for it. In the final tasks, an optimization of the system is performed for a limited range of all possible inputs, in a similar way as explained in class with the thermistor example.

The lab is divided in the tasks given in Table I:

Table I

Task	Normal task order	Required previous task	Partial punctuation
1. Schematics of a mixed signal circuit.	1	-	2.5
2. Programming the Arduino board.	2	1	2.5
3. Introducing the LCD display.	3	Task 2	2.0
4. Conditioning circuit.	4	Task 2 or Task 3	2.0
5. Results.	5	Recommended all tasks	1.0
Maximum score			10

Table II specifies the work requirements for each class group. Specifically, the table shows the sensor the group must work with and the work requirements.

Table II

Group	Sensor	Resistive load	Sensor input range
Group A	Force	10 k Ω	From 1.0 N to 5.0 N
Group B	Temperature	No required	From 10 °C to 50 °C
Group C	Flex	100 k Ω	From 0 ° to 160 °
Group D	Force	10 k Ω	From 2 N to 9 N
Group E	Temperature	No required	From 0 °C to 80 °C
Group F	Flex	100 k Ω	From 20 ° to 120 °
Group G	Temperature	No required	From 5 °C to 55 °C

Several considerations:

- The working range determines the maximum and minimum values of the physical quantity for which the output will be optimized in the last steps of this lab. It is like the example given in class.
- In the case of working with a temperature sensor, there is no need for a load resistor since the sensor has a direct voltage output.
- Flex sensors measure the amount of deflection or bending. A force sensor is used to measure weight. Since the output of both sensors is resistive, a load or pullup resistor must be used in a voltage divider assembly to transform the resistive output into voltage. It is like the thermistor case presented in class.
-

Task 1. Introducing a mixed signal circuit design.

In this task, the sensor interface of the mixed signal acquisition channel is designed. The analogue part is simulated. For it, adapt the circuit from a previous design, or create a new one using blank page.

1.1 Introduce in a Tinkercad design page the following components:

- An Arduino Uno board.
- A Breadboard Small.
- The sensor. This document is based on the TMP36 temperature sensor. If another sensor is used, the same concepts apply. Just substitute components and do following the same reasoning.
- Insert a load resistor if the sensor requires it.
 - o It is necessary in the flex and the force sensors.
- Introduce 2 voltmeters. The first one shows the power supply voltage. The second voltmeter shows the sensor output voltage. Figure 7a outlines how the circuit should be implemented.

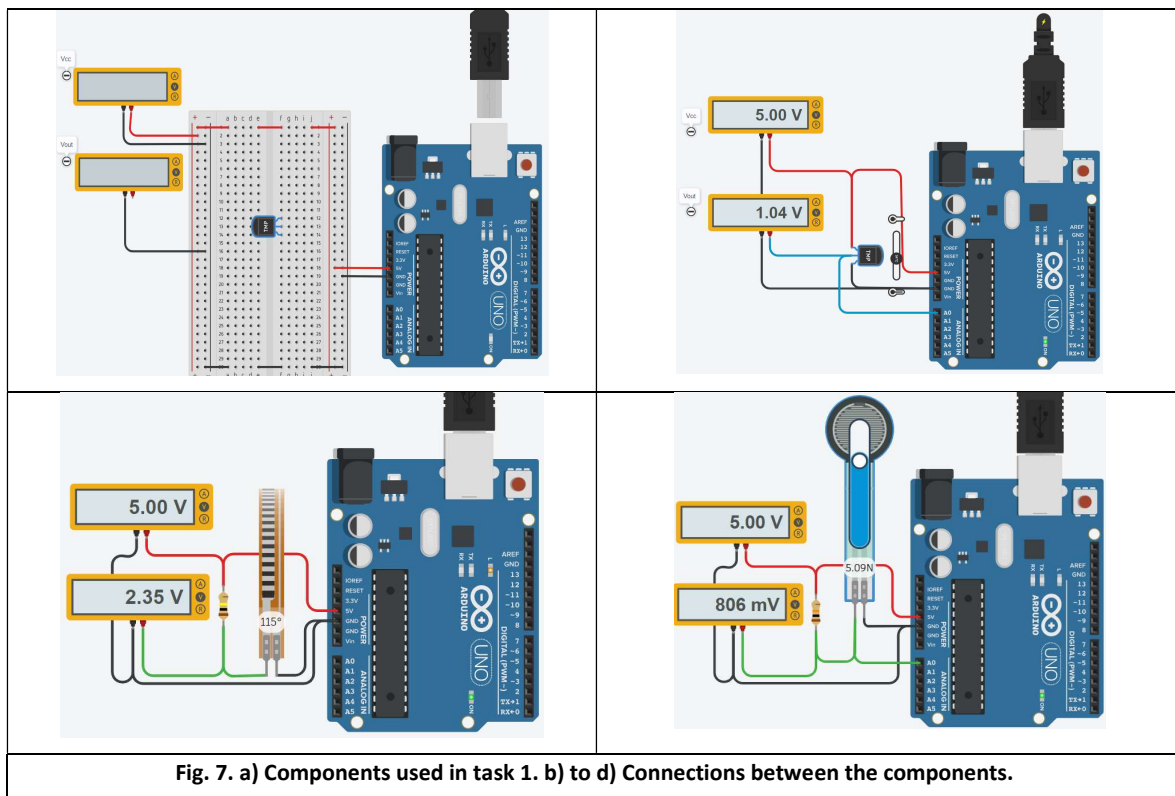


Fig. 7. a) Components used in task 1. b) to d) Connections between the components.

1.2 Connect all the components **using** the protoboard. Figures 7b to 7d show the connections for the temperature sensor, the flex sensor, and the force sensor, respectively.

Take care on connections in the breadboard:

- Power lines (+ → red, and ground → black) are connected in column.
- Rows (numbers 1, 2, ...) are connected in row. That is, connections a-b-c-d-e, and, separately, f-g-h-i-j.

1.3 Run the simulator. When touching the sensor, a slide bar appears. Move it to simulate the external data.

Note: In this first task, Arduino Uno is used only to power the circuit.

1.4 Make a *Sensor Data/Output Data* table (see Table III format) of several values where, explicitly, appears:

- The minimum and maximum values of the temperature/flex/force sensor.
- The voltage values at the specified working range in table II (the GroupTable case).
- The voltage values at the minimum and maximum temperature/flex/force values.

Note: Using flex/force sensor remember to introduce the pullup resistance to convert resistance to voltage.

In the report you must introduce:

- As many plots (screen capture) as required of the circuit realized in Task 1.
- The table *Sensor Data/Output Data* (section 1.4). Sensor data refers on temperature, bending of force, depending on the sensor. Output data is given in volts in the voltmeter.
- In the table you can fill in also intermediate values.

Table III gives a case example output format table:

	T (°C) or f (°) or F(N)	V (V)
Minimum Temp/Flex/Force value		
T0/F0		
T1/F1		
Maximum Temp/Flex/Force value		

Task 2. Programming the Arduino Uno board. A case example toggling the led.

In this part, the Arduino Uno is programmed to monitor the temperature/bending/force. Make the program using *text code*. **Do not use Codeblock because its use complicates the realization of posteriors tasks in the lab.**

2.1 Fig. 8 presents the code that implements a routine that toggles the on-board led with 100 ms of lightening and 200 ms off. Called from the main loop, and after the delay, it toggles the led during 100 ms each second.

Take the code structure given in this code as an example of good programming practice.

Copy the code, write it into Arduino program and run it. The led on the board must light during 100 ms each second.

```
// toggleLed function
// Parameters: w = width of the pulse
//             p = period of the pulse
//
void setup() {
    pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
    toggleLed(100, 200);
    delay(700);
}
void toggleLed(int p, int w){
    digitalWrite(LED_BUILTIN, HIGH);
    delay(p);
    digitalWrite(LED_BUILTIN, LOW);
    delay(w);
}
```

Fig. 8. Implementing the toggleLed function.

2.2 Use the *Arduino Serial Monitor* to print the ADC readings of the sensor. In order to do it :

- Use the *analogRead()* function to read the sensor value using the port A0.
- The variable used in *analogRead()* must be declared.
- In the setup function declare the input pin used in *analogRead()* as input.
- In the setup function, declare also the baudrate of the serial port used by the Arduino. 9600 bauds is a typical baudrate used in these case examples. 115200 bauds is the faster.
- Write the function that prints the ADC value.
- Connect the sensor output to A0 Arduino Uno pin using the breadboard circuit made in Task 1.

Starting simulation and opening The Serial Monitor window (below the writing code window) you can see the ADC conversion of the analogue input.

Fig. 9 shows part of the code and the schematics without the breadboard.

[illegible]

- As many screen captures as required of the circuit realized in Task 2.
- The algorithm implemented in section 2.3
- Following with Table III, fill in the new last three rows (Table IV):

Table IV

	$T(^{\circ}\text{C})/f(^{\circ})/F(\text{N})$	$V_{\text{out}} (\text{V})$	ADC steps	$V_{\text{INADC}} (\text{mV})$	$T_{\text{ARDUINO}}/f_{\text{ARDUINO}}/F_{\text{ARDUINO}}$
Minimum Temp/Flex value					
T_0/F_0					
T_1/F_1					
Maximum Temp/flex/Force value					

- (Optional) Comments on possible difficulties encountered in doing this part of the laboratory.

Task 3. Introducing the LCD display.

3.1 To automate the process, data shown in the monitor will be displayed in the 16x2 LCD.

For it, introduce the LCD and connect it to the Arduino board as shown in Fig. 11. With the potentiometer the LCD brightness is adjusted.

In fact, this is one of the tutorials under Autodesk Tinkercad examples. You can also find LCD information from the literature.

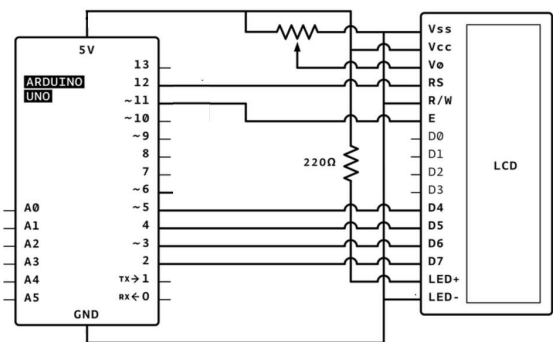
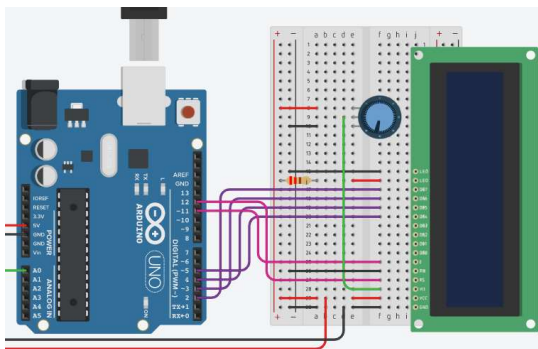


Fig. 11. Connecting the LCD to the Arduino board. Use a new protoboard to help to connect the LCD.

3.2 Introduce the C++ code to visualize an LCD message setup into the LCD.

To work with the LCD it is required:

- To initialize the ports. For example,


```
const int rs=12, en=11, d4=5, d5=4, d6=3, d7=2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);
```
- To start the LCD in the setup function:


```
lcd.begin(<#of characters>, <#number of rows>)
```
- To print into the LCD. Several functions are required to position the cursor in the LCD (check the examples in Autodesk Tinkercad and in the *Arduino Reference* web.
For example, the function `lcdInit()` in Fig. 12 can be used to initialize the LCD with a welcoming message calling it from the Arduino *setup* function.

```
void lcdInit(void){
  lcd.begin(16,2);
  lcd.print(String("Welcome "));
  lcd.setCursor(0,1);
  lcd.print("TEMP MEASURE");
  delay(1000);
}
```

Fig. 12. Welcoming LCD function.

3.3 Introduce an LCD printing function `void lcdPrint(long adc, long mVs, long temp/bending/force)` that prints the sensor monitoring into the LCD.

- `adc` corresponds to the steps in the ADC conversion. It corresponds to column *ADC steps* in Table IV.
- `mVs` corresponds to the sensor output voltage (in mV). It corresponds to column V_{INADC} in Table IV.
- `temp/bending/force` is the sensed value of the temperature/flex/force sensor, respectively. It corresponds to column $T_{\text{ARDUINO}}/f_{\text{ARDUINO}}/F_{\text{ARDUINO}}$ in Table IV.

Fig. 13 shows an output example.

Remember that only 2 columns of 16 characters are available in the LCD. A potentiometer is necessary to control the contrast in the display.

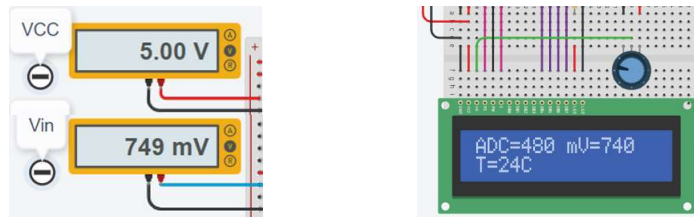


Fig. 13. Output LCD example.

To introduce in the report:

- (Optional) Comments on possible difficulties encountered in doing this part of the laboratory.
- A screen capture of the circuit realized for part 3.1.
- The Arduino Uno algorithm (sections 3.2 and 3.3).

Task 4. Conditioning circuit.

Note: Though this task can be done after Task 2, it is recommended to do it after Task 3, because the LCD introduced helps to visualize the results.

Lecture Unit 3A, Section 4 discusses the optimization of the conditioning circuitry in a thermistor circuit. It maximizes the analogue input range into an ADC in order to maximize the resolution. In the same way, this task optimizes the input of the ADC inside Arduino Uno from 0.25 to 4.75 V.

Table II shows the sensor temperatures/bending/force input range to consider for each group. For example, taking the specifications of Group A, the inlet temperatures to be considered are between 0 °C and 80 °C. This input temperatures range must be converted by the non-inverting amplifier to an ADC voltage input between 0.25 V and 4.75 V.

4.1 Design of the circuit.

Introduce in the schematics the conditioning circuit using an operational amplifier in non-inverting mode, like the circuit presented in lecture Unit 3A, Section 3.

The OpAmp is introduced using the OpAmp 741 you can find in the component library of Autodesk Tinkercad tool. Check the inputs/outputs of the device to correctly connect it into the circuit.

4.2 Make the corresponding connections.

4.3 The values of the three resistors R_h , R_l and R_f must be calculated. Introduce them in the value parameter of each resistor. Voltage reference V_{ref} is 5 V.

4.3 Simulate the circuit. Verify the output values in respect to the sensor data. To help visualizing the results it is recommended to introduce a voltmeter to visualize the output of the OpAmp.

Fig. 14 shows the components that should be used in this task:

- Protoboards
- Arduino Board
- Liquid crystal display or LCD.
- A potentiometer for adjusting the LCD screen light.
- 4 resistors. 1 in the LCD and 3 in the conditioning circuit.
- The TMP36 temperature sensor. Perhaps you need to substitute it by the flex sensor and pullup resistor.
- The 741 OpAmp.
- Three monitoring voltmeters.

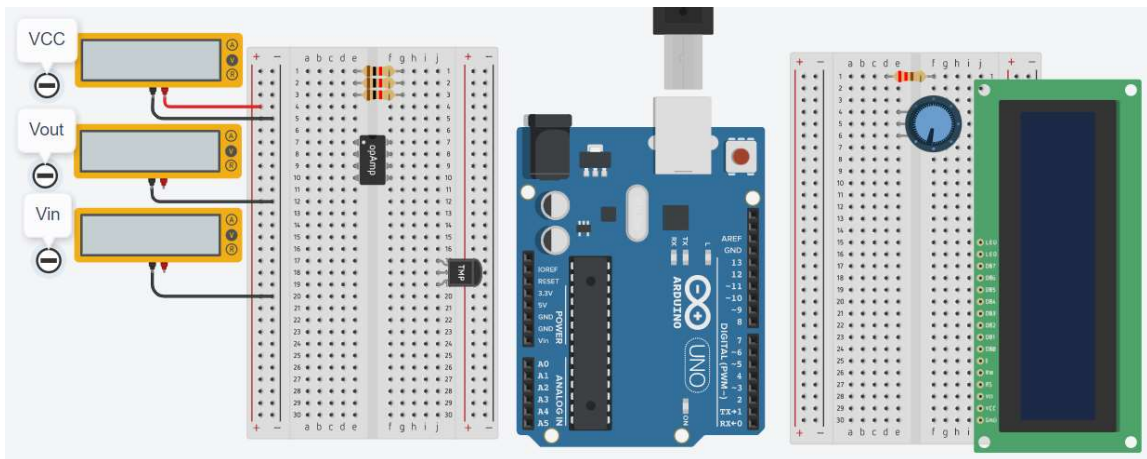


Fig. 14. Components used in the circuit. In case of the flex sensor you must also introduce the pullup resistor.

To introduce in the report:

- A screen capture of the circuit showing data and results in simulation.
- The calculus of the resistance values.
- The program written in the Arduino Uno.
- A discussion of the results obtained. Print or plot (screen capture) results based on the LCD screen or the Arduino Uno Serial Monitor.
- (Optional) Comments on possible difficulties encountered in doing this part of the laboratory.

Task 5. The sensor data acquisition system

Once all the previous tasks are done, perform a characterization of the circuit. The LCD will print the following information (Fig. 15):

- The steps converted in the ADC: ADC.
- The voltage to be converted by the ADC, corresponding to the amplifier output VA (in mV): VA
- The voltage to be amplified that it is the sensor output (in mV): Vs
- The sensor magnitude calculated in the Arduino Uno: T.



Fig. 15. To print in LCD. Substitute T field by F field if it I the case.

In the end, fill in Table V. Consider at least 8 values between the minimum and maximum input sensor range. If Task 3 has not been done, data can be obtained from the Arduino Monitor window.

Note: In case of using the flex/force sensor substitute all temperature references by flex/force values.

Table V

Temperature (°C) (read in board sensor)	Vout Sensor (mV) (read in sensor voltmeter)	Vout OpAmp (V) (read in OpAmp voltmeter)	ADC counts (ADC data in LCD)	OpAmp Output (mV) (VA in LCD)	Sensor Output (mV) (Vs in LCD)	Measured T (°C) (T in LCD)	Error (%)
Measurements obtained from the circuit board			Measurements obtained from the Arduino				
T0							
T0+5							
...							
T1							

Note 1: To and T1 are the minimum and maximum temperatures, respectively, in the input temperature range requirement.

Note 2: Error is computed between *Vout sensor* (in the voltmeter) and the calculated *Sensor output* shown in the LCD. It is given by the equation $Error\ (%) = 100 \cdot |(VoutSensor - SensorOutput) / VoutSensor|$

In the table, the first three columns correspond to data shown in the schematics, while the last columns are the calculations performed in the Arduino Uno board.

To introduce in the report:

- Screen captures at simulation of all the circuit at the temperatures T_0 and T_1 , showing the values in the voltmeters and the serial monitor. Fig. 16 gives an example.
- Table V filled in.
- The Arduino Uno program (it can be also a screen capture).

Fig. 16 shows an example of a screen capture of a case example with the thermistor sensing 25°C . The LCD shows the values.

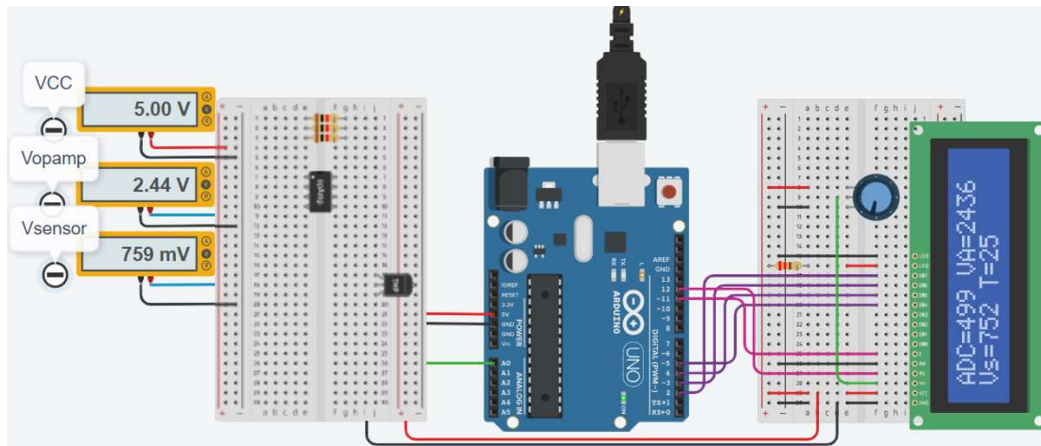


Fig. 16. Screen capture output example

3. In the report.

- Clearly indicate team members.
- Introduce in the report all the information demanded in the tasks.
- Introduce a section evaluating the participation of members in carrying out the practice.
- Send the report in a **pdf** file.

4. Good practice code.

Circuit design considerations:

- Power and ground between Arduino board and protoboard must be connected. For it, use the power supply 5 V given by the Arduino board.
- Power supply colour coding: Remember that:
 - o Power (5 V): **red colour**.
 - o Ground: **black colour**.
 - o Other colours can be used to separate parts. For example, analogue signals, digital signals, metering signals, etc.
- To draw connections elegantly. If they can go at right angles, the design is clearer.
- In the protoboard try do not cover holes by passing cables as the connection is not yet visible.

Programming considerations.

- Remember that there are two main routines:
 - o The *setup* routine, which is where the devices used are started, and where you only pass when entering to run the program.
 - o The *loop* routine that is where the program is placed and that is continuously repeated.
- In good programming, a simple main routine (the *loop* routine) is realized, creating the functions that are needed.

- Take care of the type of signal used (characters, integers, real). For example, integers are 16 bits wide, so their range is very limited.

References and helping:

Autodesk Tinkercad webpage: <https://tinkercad.com>

Arduino reference webpage: <https://docs.arduino.cc/reference/en>

Arduino example: <https://docs.arduino.cc/built-in-examples/basics/AnalogReadSerial>

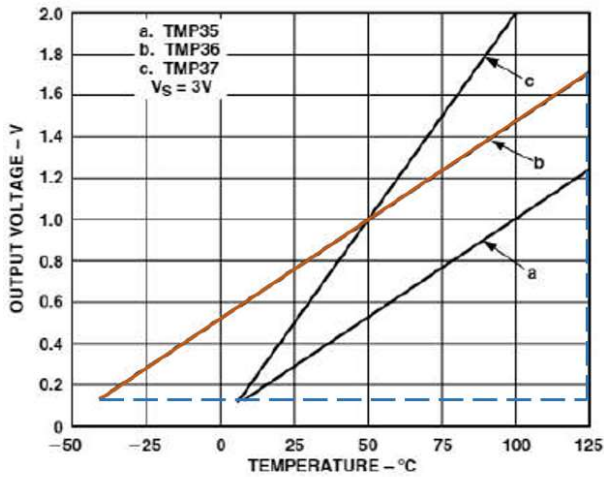
Arduino LCD webpages: <https://www.arduino.cc/reference/en/libraries/liquidcrystal/>,
<https://docs.arduino.cc/learn/electronics/lcd-displays>

Appendix: Sensor model

This appendix gives information and the model of the sensors.

Temperature sensor

From temperature sensor datasheet TMP36.pdf:



TPC 1. Output Voltage vs. Temperature

$$V_o \text{ (mV)} = V_0 \text{ (mV)} + m \text{ (mV/}^{\circ}\text{C)} * T(^{\circ}\text{C)}$$

From plot or from datasheet:

$$V_0 = 0.5 \text{ V (taken at } T = 0^{\circ}\text{C as a reference)}$$

$$m = 10 \text{ mV/}^{\circ}\text{C}$$

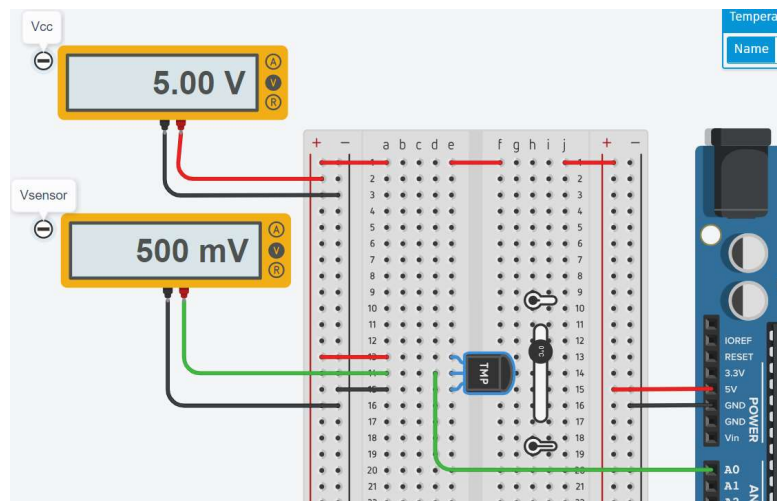
Thus,

$$V_o = 500 + 10 * T$$

V_o in mV

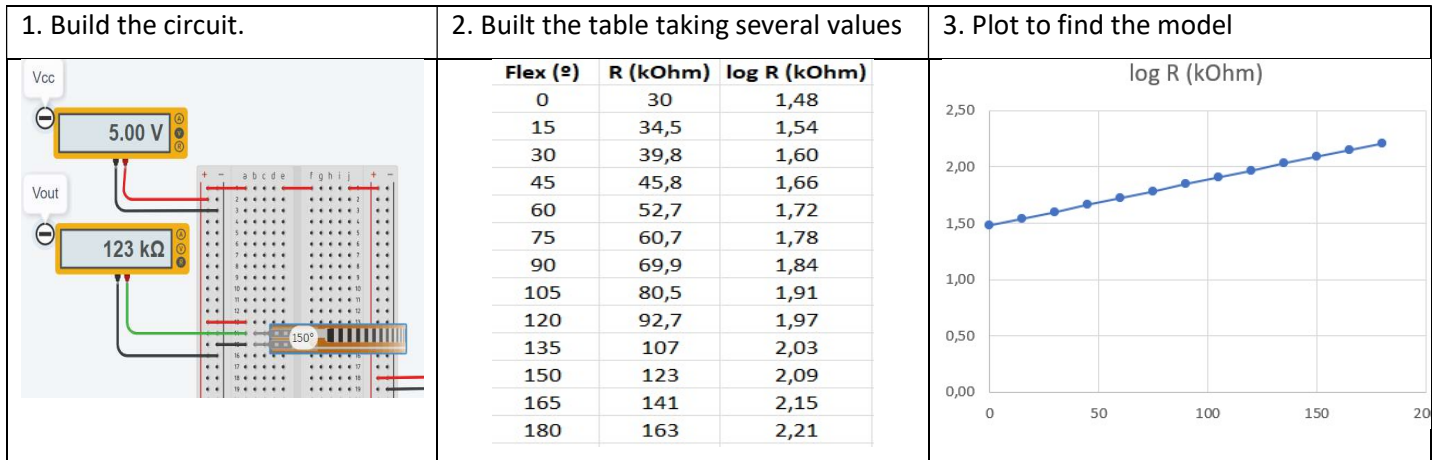
T in $^{\circ}\text{C}$

As the output of the sensor is directly in voltage, no load resistor is necessary.



Flex sensor

Tinkercad does not specify the model of the flex sensor. Thus, it can be obtained experimentally. Several values can be taken measuring its resistance and, the table and the model can be obtained.



As the model is not linear, an exponential model is supposed. In this case, transfer function is given by:

$R(k\Omega) = 10^{\log R_0 + m \cdot f(^{\circ})}$ With the parameters:

Initial constant: $\log R_0 = \log R \text{ (at } 0^{\circ}) = \log 30 = 1,47712, \text{ units} = \log(k\Omega)$

Slope: $m = \frac{2,21 - 1,48}{180 - 0} = 0,004084, \text{ units} = \log(k\Omega)/^{\circ}$

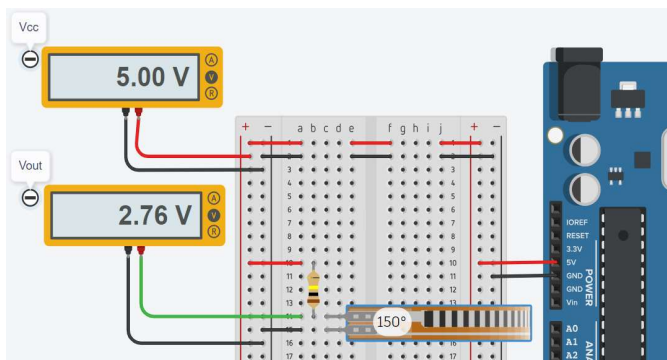
Thus, model is given by the equation:

$$R(k\Omega) = 10^{1,47712 + 0,004084 \cdot f}$$

Flexion f units are $^{\circ}$.

In this case, the sensor needs a pullup resistor to convert resistance in voltage.

A $R_{pullup} = 100 \text{ k}\Omega$ can be used. Then, the analysis of the circuit is:



Applying flex model equation at, for example, $f = 150^{\circ}$:

$$R(k\Omega) = 10^{1,47712 + 0,004084 \cdot 150} = 122,95 \text{ k}\Omega$$

Then:

$$V_o = V_{cc} \cdot \frac{R_{flex}}{R_{flex} + R_{pullup}} = 5 \cdot \frac{122,95}{122,95 + 100} = 2,76V$$

The transformation of resistance to flexion is given by the inverse equation:

$$f(^{\circ}) = (\log_{10} R - \log R_0) / m, \quad R \text{ in } k\Omega$$

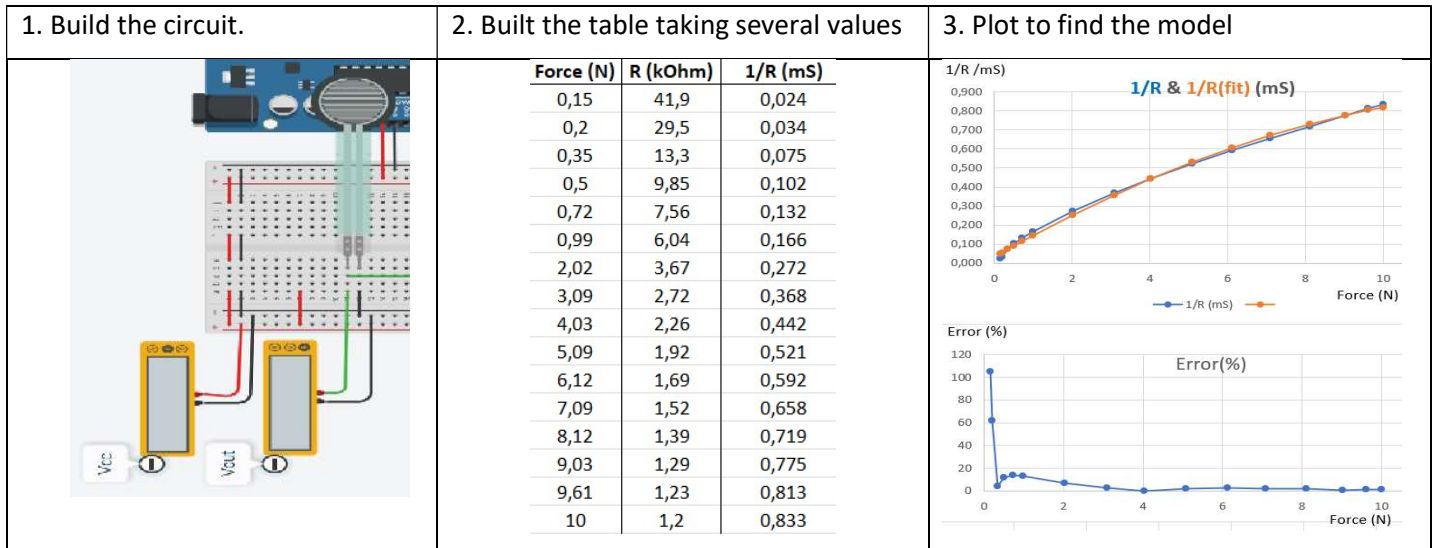
$$f(^{\circ}) = (\log_{10} R - 1,47712) / 0,004084, \quad R \text{ in } k\Omega$$

For example, applying this equation to the example:

$$f(^{\circ}) = (\log_{10} 122,95 - 1,47712) / 0,004084 = 150,002^{\circ}$$

Force sensor

Tinkercad does not specify the model of the flex sensor. Thus, it can be obtained experimentally. Several values can be taken measuring its resistance and, the table and the model can be obtained.

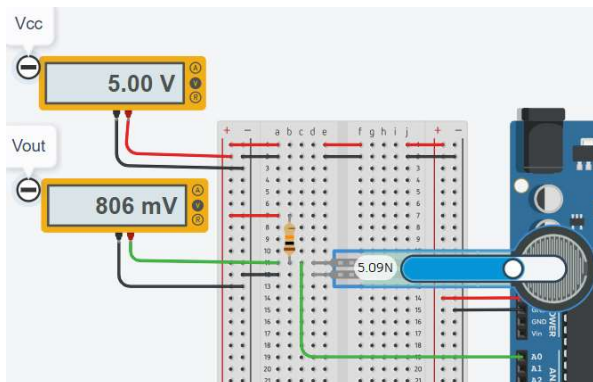


In this case a $1/R$ (in mS) followed by a second order equation fitting can model adequately the sensor behaviour for force values over 0.25 N up to 10 N. We will use this model here on.

$$1/R = -0.004 \cdot F^2 + 0.118 \cdot F + 0.031, \quad F \text{ units in N, } R \text{ in } k\Omega$$

A pullup resistor $R_{pullup} = 10 \text{ k}\Omega$ to can be used to convert resistance into voltage.

Then, the analysis of the circuit is:



Applying force model equation at, for example, 5 N:

$$1/R = -0.004 \cdot 5^2 + 0.118 \cdot 5 + 0.031 = 0.521 \text{ mS}$$

$$R = 1/0.521 = 1.92 \text{ k}\Omega$$

Then:

$$V_o = V_{cc} \cdot \frac{R_{force}}{R_{force} + R_{pullup}} = 5 \cdot \frac{1,92}{1,92 + 10} = 0.805 \text{ V}$$

The transformation of resistance to flexion is given by the inverse equation:

$$F(N) = \frac{-B \pm \sqrt{B^2 - 4A(C - 1/R)}}{2A}, \quad \begin{cases} A = -0.004 \\ B = 0.118 \\ C = 0.031 \end{cases}, \quad R \text{ in } k\Omega$$

The solution has 2 solutions. **Only the solution between 0 N and 10 N is the correct.**

For example, applying this equation to the example:

$$F(N) = \frac{-0.118 \pm \sqrt{0.118^2 + 4 \cdot 0.004(1 - 0.031/1.92)}}{-2 \cdot 0.004} = \begin{cases} 5 \text{ N, correct solution} \\ 24.5 \text{ N, incorrect solution} \end{cases}$$

* Siemens (S) unit is the inverse of the Ohm (Ω) unit.